

Hospital Automated Booking System

Project Architecture and Agent Flow

This PDF summarizes the current backend architecture, the LangGraph routing loop, and the main patient flows in the Smart Hospital Portal.

Core Architecture

- Frontend: `app/api/static/index.html`, `app.js`, and `styles.css` render the portal UI and send chat requests.
- API layer: `app/api/main.py` exposes `auth`, `chat`, `appointments`, and `admin` routers.
- Chat route: `app/api/routes/chat.py` loads the patient profile, history, active appointments, and executes the agent graph.
- Agent graph: `app/agents/graph.py` runs the LangGraph workflow and keeps the conversation memory compacted.
- Supervisor router: `app/agents/supervisor.py` decides the next specialist or ends the turn.
- Specialist agents: `triage_router`, `conversation_agent`, `remedy_agent`, `medical_rag`, and `appointment_booker` handle the medical and booking steps.
- Services: `app/services` and `app/db` provide chat history, appointment operations, memory policy, embeddings, vector search, token tracking, and persistence.

Primary Runtime Flow

Frontend -> POST /chat or /chat/stream -> run_patient_chat -> LangGraph supervisor
-> specialist agent -> supervisor -> final response.

What The Supervisor Checks First

- Profile and account questions
- Chat end / close requests
- Booking lookup, cancel, or reschedule requests
- Direct booking requests
- Symptom triage or medical routing

Special Routes

- Profile query -> answer from patient_profile and bypass medical agents.
- Symptoms -> triage_router -> conversation_agent -> remedy_agent.
- Persisting symptoms -> medical_rag -> appointment_booker.
- Unknown department -> medical_rag before booking.
- Booking lookup, cancellation, reschedule -> appointment_booker.
- End chat -> end_confirmation -> finish.

Agents and Responsibilities

- Supervisor: decides whether the message continues the current flow or diverts to a specialist.
- Triage router: extracts intent, symptoms, and severity from the latest message.
- Conversation agent: asks follow-up intake questions until enough context is collected.
- Remedy agent: generates personalized care tips and handles follow-up about improvement or persistence.
- Medical RAG: maps symptoms and context to the most relevant department using heuristics, vector search, and LLM reasoning.

State That Drives Routing

Appointment Booker: handles doctor selection, slot selection, booking confirmation, rescheduling, and cancellation.
awaiting, intent, symptoms, severity, target_department, remedy_given, persisting, booking_active, confirmed_booking, confirmed_bookings, chat_closed, and patient_profile.

Important Bypass Scenarios

- Chat already closed: `run_patient_chat` returns a closed-chat response immediately.
- User asks for profile info: the supervisor answers from `patient_profile` and skips the medical flow.
- User asks for a booking lookup: the supervisor returns the active or past bookings summary.
- User asks for a remedy during booking: the booking state is cleared and the flow returns to `remedy_agent`.
- RAG cannot confidently match a department: the flow returns to `conversation_agent` for more intake.

Data Stores and Supporting Services

- Emergency severity: the remedy step is bypassed in favor of emergency guidance.
- Chat history and session summaries are persisted by `app/services/chat_history.py` and `app/services/llm_usage.py`.
- Appointments use `app/services/appointments.py` and the routes in `app/api/routes/appointments.py`.
- Medical department matching uses `app/services/rag.py` with vector search and reranking.
- Database schema and rebuild scripts live under `app/db`.

API Surface and Functions Used

- Auth API: POST /auth/signup, POST /auth/login, POST /auth/admin/login, GET /auth/me, PATCH /auth/profile.
- Chat API: POST /chat, POST /chat/stream, GET /chat/history, POST /chat/upload, POST /chat/confirm-processing, GET /chat/document-status/{id}.
- Appointment API: GET /appointments/departments, /doctors, /slots, /available, /upcoming, /previous, POST /book, /{id}/cancel, /{id}/reschedule, GET /{id}/reschedule-options.
- Admin API: analytics, doctors, slots, holidays, and appointment dashboard endpoints under /admin.
- WebSockets: /ws/status/{session_id} for document-ingestion progress and /chat/deepgram for live audio bridging.

Main Functions

- app/api/main.py: FastAPI app creation, lifespan startup, health check, static file serving, and WebSocket connection manager.
- app/api/routes/chat.py: _prepare_chat_state, _run_chat_with_usage, chat, chat_stream, upload_document, confirm_processing, document_status.
- app/agents/graph.py: initialise_hybrid_memory, compact_hybrid_memory, arun_patient_chat, run_patient_chat.
- app/agents/supervisor.py: supervisor_node, route_from_supervisor, continue_current_node, general_qa_node, routing heuristics.
- app/agents/triage_router.py: triage_router_node and triage streaming helpers.
- app/agents/conversation_agent.py: conversation_agent_node, conversation_agent_stream, finalize_conv_stream_state.
- app/agents/remedy_agent.py: remedy_agent_node.
- app/agents/medical_rag.py: medical_rag_node.
- app/agents/appointment_booker.py: appointment_booker_node and booking-menu helpers.
- app/agents/document_analyzer.py: document_analyzer_node.

How The App Handles 100000 Users

- Short answer: not safely as-is. The code is structured well, but it is not yet production-hardened for 100000 truly simultaneous users.
- Current bottlenecks: `psycopg2.connect()` creates a fresh blocking DB connection per call; the app does not show an async database pool; LLM calls and vector searches can become latency hotspots; document extraction is CPU-heavy; and LangGraph checkpoints are stored in SQLite, which is not ideal for massive concurrent writes.
- What already helps: conversation memory is compacted, recent-history windows are small, static prompts are reused, booking lookups are limited, and session state is namespaced by `patient_id` + `session_id` to reduce leakage.

- What would be needed for 100k scale: a real PostgreSQL connection pool,

async DB access or worker isolation, Redis or another external session/cache

layer, separate queues for document ingestion and long LLM jobs, horizontal

stateless API replicas behind a load balancer, and rate limits with

shared Redis-backed pool.

backpressure.

2. Move SQLite checkpoints to PostgreSQL or Redis for multi-replica

durability. booking safety: slot booking must stay transactional with unique

constraints and row locks so two users cannot reserve the same slot at once.

3. Put document analysis and any long-running LLM work on background workers.

4. Add Redis-backed rate limiting, per-user quotas, and request timeouts.

Operational reality: 100000 simultaneous browser sessions is different from

100000 active LLM calls. The app can support many idle sessions only if chat

state is kept out of process and expensive work is throttled.

5. Cache department lookups, doctor lists, and common prompts where safe.

6. Load test booking writes separately from read-heavy chat traffic.

Data Stores, Integrations, and Runtime Notes

- PostgreSQL stores users, patient profiles, doctors, slots, holidays, bookings, chat history, admin data, token logs, and document catalog rows.
- Qdrant stores the clinical knowledge base that powers department matching and medical RAG.
- Azure OpenAI is used for chat generation, routing, summary generation, document relevance checks, and document extraction.
- Azure Blob Storage stores staged uploads, vault files, and generated JSON summaries.
- Sentence Transformers provide embeddings for vector search, and optional CrossEncoder reranking improves department matching.

Key Runtime Safeguards

- Nginx serves the frontend and reverse-proxies API and WebSocket traffic.
- JWT authentication is required for patient actions.
- Profile data and bookings are reloaded from the database each turn instead of trusting client state.
- Chat state uses patient_id plus session_id thread IDs to reduce cross-user leakage.
- Document uploads are size-limited, MIME-filtered, and verified before staging.
- Booking queries respect a 30-minute minimum lead time and a 7-day lookahead window.